LUMS
A Not-for-Profit University

School of Science and Engineering

AI624 AI on Edge Devices

ASSIGNMENT 3 – Quantization

Due Date: 11:59 PM, 23rd October.
Format: 3 tasks worth a total of 100 marks.
Instructions:

- Collaboration with peers is allowed, but copying another student's solution is strictly prohibited. This is not a group assignment—each student must submit their own work.
- Submit all scripts/notebooks along with visible results, as well as a comprehensive Markdown file summarizing the results and analysis for each task and subtask.
- Submit one ZIP file containing a main Jupyter notebook with all outputs visible, any supporting Python modules, and a Markdown report summarizing results and analysis for every task. Present results in tables where appropriate and equations in LaTeX. Include dependencies, execution instructions, and a brief codebase outline in a README.
- Submit your work on LMS by the deadline.
- Start early. This is a lengthy assignment, and we do not expect you to complete it in the last few days.
- If you have any concerns, contact the instructor or teaching assistants. You are also encouraged to use the assignment Slack channel for questions and discussion.

- You are among the most competent individuals in the country. Do not let plagiarism undermine your learning. Any instance of plagiarism will be dealt with according to the university's rules and regulations.
- You must acknowledge any use of generative AI tools. Include the following statement where applicable:

  I have used [insert Tool Name] to [write, generate, plot, or compute, explaining the specific use of generative AI] [number of times].
- You may optionally share a link to the relevant chatbot conversation for further clarity.

Implementation Requirements

- **Implement the quantization techniques yourself.** Calling ready-made SmoothQuant, GPTQ, fake-quantization, or QAT preparation functions from external packages is not allowed. Wrapping or copying their quantization implementation does not satisfy this requirement. You may study papers and reference code with attribution. Standard model loading, tokenization, dataset loading, tensor operations, linear algebra, and profiling utilities are allowed.
- Tasks 1 and 2 use pretrained GPT-2 small without training. Fine-tuning is permitted only for the ResNet-18 QAT experiments in Task 3.
- Within each task, use the same checkpoint, data processing, data splits, and profiling workload for all comparisons. Select hyperparameters using validation data only. Record software versions, random seeds, and reproduction commands.

Experimental Setup

Use **GPT-2 small**, approximately 124 million parameters, from the pretrained checkpoint `openai-community/gpt2`. The original model is introduced by Radford et al. [1]. OpenAI's official repository provides the original model and code [2]. For a PyTorch implementation suitable for this assignment, use the Hugging Face Transformers GitHub repository and pretrained checkpoint [3, 4].

Use **WikiText-2**, the `wikitext-2-raw-v1` configuration of `Salesforce/wikitext` [5]. Fix the input sequence length to **128 tokens**.

- Use 128 sequences sampled from the training split for calibration. Record the seed and reuse exactly these sequences for all PTQ methods.
- Use the validation split for hyperparameter selection and the test split for final accuracy and perplexity. Never calibrate or tune using the test split.
- Tokenize with the GPT-2 tokenizer. Concatenate nonempty examples within each split using the end-of-text token as a separator, then form nonoverlapping blocks of 128 tokens. Discard the final incomplete block and report how many tokens are discarded.
- Score the 127 next-token targets within each block. Do not score the first token without context. Use the identical blocks and target positions in every experiment.
- Use batch size 1 for profiling. Evaluation and calibration may use a larger batch size if memory permits, but keep it fixed across methods and report it.
- Use FP32 for the unquantized baseline and for operations retained in floating point. Run inference in evaluation mode with gradients disabled. Set `use_cache=False` for the fixed-length forward-pass experiments.

1 Task 1: GPT-2 Baseline and Model Profiling [20 marks]

1.1 Model Architecture and Quantization Scope [5 marks]

GPT-2 is an autoregressive, decoder-only Transformer that predicts the next token from preceding tokens. GPT-2 small contains learned token and positional embeddings, 12 Transformer blocks, a final LayerNorm, and a vocabulary projection tied to the token embedding weights. Its hidden dimension is 768, with 12 attention heads and an MLP intermediate dimension of 3072 [1, 3, 4].

Each block contains pre-LayerNorm causal multi-head self-attention, an attention output projection, an MLP with GELU, and residual connections. Briefly explain these components and show the order of operations within one block.

The following projection operations will be quantized in Task 2 in **all 12 blocks**.

Projection	**GPT-2 module**
Query, key, and value	`transformer.h[i].attn.c_attn`
Attention output	`transformer.h[i].attn.c_proj`
MLP input	`transformer.h[i].mlp.c_fc`
MLP output	`transformer.h[i].mlp.c_proj`

Quantize each listed projection's **weights and input activations**. Keep its bias in FP32. Keep embeddings, the tied vocabulary head, LayerNorm, GELU, softmax, residual additions, and the attention-score and attention-value matrix multiplications in floating point.

Do not quantize the KV cache. Cached keys and values, if used in any supplementary generation experiment, must remain in floating point to avoid introducing cache quantization as an additional source of generation error. The required experiments use full 128-token forward passes without caching.

GPT-2's projection modules may use the Transformers `Conv1D` class. These are linear projections with a weight layout that differs from `nn.Linear`. Verify input and output dimensions before choosing quantization axes or replacing a module.

1.2 Baseline Language-Model Performance [5 marks]

Evaluate the original pretrained model on the WikiText-2 test split. Report **next-token Top-1 accuracy**, mean negative log-likelihood (NLL), and perplexity. Accuracy is the fraction of scored positions for which the highest-probability vocabulary token matches the ground-truth next token, using the ground-truth prefix.

For the set of scored targets $\mathcal{T}$ and $N = |\mathcal{T}|$,

$$\text{Accuracy} = \frac{100}{N} \sum_{t \in \mathcal{T}} \mathbf{1}\left[\arg\max_{v} p(v \mid x_{<t}) = x_t\right],$$

$$\text{NLL} = -\frac{1}{N} \sum_{t \in \mathcal{T}} \log p(x_t \mid x_{<t}), \qquad \text{PPL} = \exp(\text{NLL}).$$

Aggregate loss over target tokens rather than averaging batch perplexities. Report the total number of scored targets. Perplexity is the primary language-model metric, with accuracy providing a complementary comparison.

1.3 Baseline Profiling [10 marks]

Use a profiling tool such as PyTorch Profiler [8]. Profile the baseline using 128-token inputs and the fixed profiling batch size. Use at least 10 warm-up passes and 30 measured forward passes. Synchronize the GPU when timing GPU execution. Measure latency separately from profiler instrumentation and exclude model loading and tokenization.

Report the following metrics for the baseline and repeat the same measurements for every Task 2 configuration.

- **Model storage:** unique parameter count and serialized model size in MB. Count tied parameters once.

- **Computational cost:** MACs and FLOPs per forward pass, with a breakdown for attention projections, attention matrix multiplications, MLP projections, and the vocabulary head. State the counting convention. Use one MAC as two arithmetic FLOPs and identify any additional counted operations. Supplement profiler counts analytically when operators are not covered.
- **Runtime:** mean and standard deviation of latency in milliseconds, sequences per second, and input tokens processed per second.
- **Memory and utilization:** peak GPU allocated memory, peak process CPU memory, and average CPU and GPU utilization where available. State the measurement tools and distinguish allocated GPU memory from total device memory.
- **Power and energy:** average CPU and GPU power in watts and energy per forward pass in millijoules where supported. Identify the sensor, sampling interval, and whether the measurement covers the process or the entire device.
- **Execution environment:** CPU and GPU models, device memory, framework versions, and the device used for inference.

Estimate energy by integrating measured power over repeated forward passes. Extend the measurement interval if needed to obtain several power samples. For K passes over interval T,

$$E_{\text{forward}} = \frac{1}{K}\int_0^T P(t)\,dt, \qquad E_{\text{MAC}} = \frac{E_{\text{forward}}}{\text{MACs}_{\text{forward}}}, \qquad E_{\text{FLOP}} = \frac{E_{\text{forward}}}{\text{FLOPs}_{\text{forward}}}.$$

Report average energy per MAC and per counted arithmetic operation in appropriate units. These are whole-forward-pass averages that include memory and execution overhead, rather than direct measurements of individual instructions. Report CPU and GPU energy separately when both are available. If a sensor or utilization metric is inaccessible on the selected platform, mark it unavailable and briefly state the limitation.

2 Task 2: GPT-2 Post-Training Quantization [60 marks]

Start every method from the **same original pretrained checkpoint**. SmoothQuant and GPTQ are separate comparisons against linear PTQ, not a combined pipeline.

For **each method**, evaluate the following four configurations.

Configuration	Weight precision	Activation precision
Static W8A8	8 bits	8 bits, fixed calibration range
Dynamic W8A8	8 bits	8 bits, runtime range
Static W4A8	4 bits	8 bits, fixed calibration range
Dynamic W4A8	4 bits	8 bits, runtime range

Use symmetric **per-output-channel weight quantization** and asymmetric **per-tensor activation quantization** throughout. Fix weight scales before inference. Static A8 uses one fixed range per projection input from calibration. Dynamic A8 recomputes the range over the current projection-input tensor on each forward pass. Keep granularity identical across methods so that comparisons isolate the quantization technique.

W8A8 and W4A8 refer to the listed projection operations. They do not imply that the entire model executes at that precision.

2.1 Static and Dynamic Linear PTQ [20 marks]

Implement uniform round-to-nearest quantization for weights and activations. For a real-valued element r,

$$q = \text{clip}\left(\text{round}(r/S) + Z, q_{\min}, q_{\max}\right), \qquad \widehat{r} = S(q - Z).$$

For a b-bit symmetric weight quantizer, use the integer range $[-Q_b, Q_b]$, where $Q_b = 2^{b-1} - 1$, and

$$S_W^{(c)} = \frac{\max_i |W_{ic}|}{Q_b}, \qquad Z_W^{(c)} = 0.$$

Here c indexes the output channel in the row-vector convention $Y = XW + b$. For unsigned A8, include zero in the observed activation range and use

$$S_X = \frac{x_{\max} - x_{\min}}{255}, \qquad Z_X = \text{clip}\left(\text{round}(-x_{\min}/S_X), 0, 255\right).$$

Handle zero-range tensors safely. Use these quantizer conventions in all later subsections.

1. Implement weight quantization, activation observers, static calibration, and dynamic range estimation. For static calibration, aggregate the minimum and maximum over all calibration inputs to each projection.
2. Derive the quantized projection expression, including integer accumulation, activation zero-point correction, output rescaling, and FP32 bias addition.
3. Evaluate all four configurations. Report test accuracy, NLL, perplexity, and every profiling metric required in Task 1 if applicable as it depends on the GPU selected and its available integer multipliers which may or may not contain INT8 and INT4 multipliers. Include scale and zero-point storage in the serialized size. If the latter is the case, do a theoretical analysis on whether or not there is a reduction in MACs/FLOPS, memory requirements for computation, average energy per MAC, and average energy per FLOP.
4. Measure layer-wise normalized weight and activation reconstruction error on identical validation inputs. Explain differences between static and dynamic results at each precision.

For a tensor A, use

$$E_A = \frac{\|A - \widehat{A}\|_F^2}{\|A\|_F^2 + \epsilon}.$$

2.2 SmoothQuant and Outlier Analysis [25 marks]

Xiao et al. introduce *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models* [6]. The method uses an equivalent channel-wise scaling transformation to reduce activation outliers by transferring part of the quantization difficulty to weights. The original method targets W8A8. Its W4A8 application here is an experimental extension to analyze if we can observe gains at even lower precision of weights.

Before smoothing, inspect both weight and input-activation distributions for the attention and MLP projections. For representative blocks 0, 5, and 11, provide histograms and plots of per-input-channel maximum absolute magnitude. Report maximum absolute value and the 99th percentile of absolute values. Identify whether a few channels dominate the activation range and whether weights show comparable outliers.

For $Y = XW + b$, introduce a positive diagonal matrix $D = \text{diag}(s)$ such that

$$X' = XD^{-1}, \qquad W' = DW, \qquad X'W' + b = XW + b,$$
$$s_j = \frac{(\max |X_{:j}| + \epsilon)^{\alpha}}{(\max |W_{j:}| + \epsilon)^{1-\alpha}}.$$

The activation maxima are estimated only from calibration data. The parameter α controls how much quantization difficulty is transferred from activations to weights.

1. Implement the channel-scale computation and equivalent transformation yourself. Explain where the scales are applied or folded into GPT-2. Preserve equivalence for all branches sharing an input. Explicit runtime scaling is acceptable if it is included in profiling.

2. Verify floating-point equivalence before quantization. Report maximum absolute output differences and relative output error on fixed inputs.

3. Compare $\alpha \in \{0.25, 0.5, 0.75\}$ using validation perplexity. Select and report α for each required configuration. Recalibrate static activation ranges after smoothing.

4. Quantize the transformed weights and activations using the conventions in Section 2.1. Evaluate all four SmoothQuant configurations and report accuracy, NLL, perplexity, and all Task 1 profiling metrics if applicable.

5. Repeat the distribution analysis on the same validation sequences after smoothing. Compare weight and activation ranges, outliers, clipping fractions, and reconstruction errors against the corresponding static and dynamic linear PTQ cases.

For distribution comparisons, show the pre-quantization tensor and its quantized/dequantized approximation. Changing a static quantizer to a dynamic quantizer does not itself transform the original tensor distribution. Distinguish range-selection effects from SmoothQuant's scaling transformation.

Analysis. If SmoothQuant improves accuracy or perplexity, explain why using the observed channel ranges, activation reconstruction errors, and corresponding weight errors. Discuss the trade-off introduced by α and whether the benefit changes between W8A8 and W4A8 or between static and dynamic A8. If results do not improve, explain them using the same evidence. Relate runtime and energy changes to the actual execution path rather than bit-width alone.

2.3 GPTQ and Reconstruction Analysis [15 marks]

Frantar et al. introduce *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers* [7]. GPTQ uses approximate second-order information from calibration inputs to quantize weights while compensating for the errors introduced in previously quantized weights. Its aim is to preserve layer outputs more accurately than independent round-to-nearest quantization.

GPTQ quantizes weights. To obtain W8A8 and W4A8 in this assignment, combine GPTQ-quantized W8 or W4 weights with the separate static or dynamic A8 quantizer from Section 2.1.

For calibration input matrix X, the reconstruction objective and a Hessian approximation are

$$\min_{\widehat{W}} \|XW - X\widehat{W}\|_F^2, \qquad H = 2X^{\top}X + \lambda I.$$

1. Inspect weight and activation distributions before quantization using the same blocks and statistics as Section 2.2. Use the original pretrained model without SmoothQuant scaling.

2. Implement calibration-input collection, Hessian accumulation, damping, sequential weight quantization, and compensation of errors in the remaining unquantized weights. Implement the block-wise processing strategy described in the paper. Process layers sequentially using calibration inputs from the already quantized preceding layers.

3. Explain how the Hessian or its inverse guides error compensation. Report the damping rule, processing block size, quantization ordering, and numerical stabilization. Retain the same per-output-channel weight scales and integer grid as the matching linear PTQ baseline.

4. Evaluate GPTQ with static A8 and dynamic A8 at both W8A8 and W4A8. Calibrate static activation ranges using the GPTQ-weight model. Report accuracy, NLL, perplexity, and every profiling metric from Task 1. Also report quantization time and peak memory during GPTQ calibration and reconstruction.

5. Compare layer-output reconstruction error and weight quantization error against round-to-nearest PTQ on identical validation inputs. Repeat the activation error and clipping measurements after applying A8 quantization.

Analysis. If GPTQ improves accuracy or perplexity, explain how calibration-aware error compensation preserves layer outputs. Discuss why smaller individual weight errors are not the sole objective and why activation quantization can still limit the final result. Compare against linear PTQ and SmoothQuant at matching precision and activation settings. Support any claimed advantage with measured errors and profiling results. Improvement is not required for full credit if the implementation and analysis are correct.

2.4 Consolidated Results

Include one comparison table with the FP32 baseline and the 12 quantized configurations. Report accuracy, NLL, perplexity, model size, MACs, FLOPs, latency, throughput, memory, and available utilization and energy metrics if applicable. Put detailed layer-wise results and distribution plots in separate clearly labeled tables or figures. Summarize which method performs best at each precision and explain the main accuracy, storage, runtime, and energy trade-offs.

Execution and storage reporting. State whether each configuration uses genuine integer kernels or simulated quantize-dequantize operations around floating-point kernels. Either is acceptable, but simulated execution must be labeled clearly. Explain added calibration, runtime range estimation, scaling, and quantization overhead. Quantization generally preserves the projection MAC count while changing arithmetic precision. Distinguish arithmetic operation counts from conventional floating-point FLOP counts for integer execution.

Report actual serialized size, including unquantized tensors, biases, scales, zero-points, and metadata. State whether 4-bit weights are packed. If the implementation stores floating-point simulated weights, report that actual storage and list the theoretical packed size separately. Do not claim a measured storage reduction or integer speedup from a theoretical bit-width change.

3 Task 3: Quantization-Aware Training on ResNet-18 [20 marks]

Apply quantization-aware training (QAT) to a pretrained **ResNet-18 on CIFAR-10** at **W8A8** and **W4A8**. Compare both final QAT models against the original floating-point model.

You may import the pretrained model and its preprocessing utilities from the `edadaltocg/detectors` GitHub repository [9], using the `resnet18_cifar10` model. Alternatively, reuse the repository and pretrained checkpoint used in PA1. Report the checkpoint source and maintain the same model and preprocessing throughout this task.

3.1 Model Setup [4 marks]

1. Load the pretrained model and establish its floating-point accuracy and profiling results.

2. Create a fixed training and validation split from the CIFAR-10 training set. Reserve the test set for final evaluation.

3. Start each QAT experiment independently from the same pretrained checkpoint. Use identical data splits and training budgets for both precisions.

3.2 QAT Implementation [8 marks]

Simulate quantization during the training forward pass while maintaining floating-point master weights for optimization. Use a straight-through estimator to propagate gradients through rounding.

1. Apply fake quantization to the weights and input activations of all convolutional and fully connected layers.
2. Use symmetric per-output-channel weight quantization and asymmetric per-tensor activation quantization. Keep biases in floating point.
3. Implement fake quantization and the straight-through estimator yourself. Explain how gradients are handled within and outside the clipping range.
4. Initialize activation observers using training data. Report how their statistics are updated and when they are frozen.
5. State how BatchNorm is handled and whether it is folded into convolutional layers.
6. Fine-tune with fake quantization enabled. Select the final checkpoint using validation performance only.
7. Freeze activation quantization parameters before final evaluation. Evaluate with fake quantization enabled or convert to a supported integer backend.

Report the optimizer, learning rate, batch size, number of epochs, augmentation, observer settings, random seed, and training time.

3.3 Results and Profiling [5 marks]

For the floating-point baseline and both final QAT models, report:

- Training and test Top-1 accuracy, test Top-5 accuracy, and test Macro-F1.
- Serialized model size, including scales, zero-points, biases, and quantization metadata.
- MACs, FLOPs, inference latency, throughput, memory usage, and available CPU and GPU utilization and energy measurements.
- Layer-wise normalized weight and activation reconstruction error for the QAT models.

Plot training and validation loss and accuracy against epochs for both experiments.

Compute reconstruction errors of the quantized/dequantized weights and activations relative to the floating-point master weights (or the original FP32 pretrained weights) before fake quantization.

Use the profiling methodology from Task 1 with CIFAR-10 images at the expected input resolution. Keep the input shape and profiling batch size fixed. State whether inference uses simulated quantization or actual integer kernels. Distinguish actual serialized storage from theoretical packed storage.

3.4 Analysis [3 marks]

Compare W8A8 and W4A8 QAT against the original floating-point baseline. Discuss final accuracy and Macro-F1, training convergence, quantization errors, storage, latency, memory, energy, and the additional training cost. Support your conclusions using the training curves and measured results.

References

[1] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[2] OpenAI. *GPT-2: Official Code and Pretrained Models*. Original TensorFlow implementation, archived repository. https://github.com/openai/gpt-2

[3] Hugging Face. *Transformers: GPT-2 PyTorch Implementation*. https://github.com/huggingface/transformers/tree/main/src/transformers/models/gpt2

[4] Hugging Face. *GPT-2 Small Pretrained Checkpoint*. https://huggingface.co/openai-community/gpt2

[5] S. Merity, C. Xiong, J. Bradbury, and R. Socher. *Pointer Sentinel Mixture Models*. ICLR, 2017. WikiText dataset. https://huggingface.co/datasets/Salesforce/wikitext

[6] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models*. ICML, 2023. https://arxiv.org/abs/2211.10438

[7] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. ICLR, 2023. https://arxiv.org/abs/2210.17323

[8] PyTorch. *Profiler Documentation*. https://docs.pytorch.org/docs/stable/profiler.html

[9] E. Dadalto. *Detectors: Pretrained Models and Preprocessing Utilities*. https://github.com/edadaltocg/detectors

ResNet-18 CIFAR-10 checkpoint: https://huggingface.co/edadaltocg/resnet18_cifar10